01 / ROLE AND ROADMAP

Sudeshhna's Phase 1 work guide

PROJECT 11 / AUTOMATIC CLOTHING CATEGORY RECOGNITION

Inventory Image Tagging via Handcrafted Features and a Small CNN
Phase 1 focus: classical machine learning on Fashion-MNIST.

Your role: Data and EDA lead

Load the official data, confirm labels, and save one shared split.

The five-person division

Member	Primary ownership
Nakul	Integration, literature and report Repository, master notebook, report assembly
Sudeshhna	Data quality and EDA Loader, fixed split, audit, EDA figures
Puneet	Features, Logistic Regression and kNN Pixel/HOG features and four model rows
Samarth	Random Forest, SVM and run harness Four model rows and experiment logs
Neelanshu	Evaluation and error analysis Metrics, timing, complexity, error figures

How to use this guide

Read pages 2-4 first, then work through your practical pages in order. Pages 13-16 explain handoffs, viva preparation, the proposed schedule and sources. Your guide is a work plan and learning manual, not a completed experiment or a submission-ready report.

All five members must understand the entire pipeline. Ownership means you implement and explain an area; it does not mean the others can skip learning it. Each member also writes their report section, prepares their slides, and reviews another member's work.

Prepared 27 October 2026. All workload estimates, code organization, hyperparameter grids and internal milestones are recommendations, not faculty rules. No model scores have been invented.

02 / SHARED FOUNDATIONS

What Phase 1 actually requires

The supplied CSAI instructions are the course requirements. Your project description supplies the technical problem. This guide adds a practical way to satisfy both; it does not turn its suggestions into extra faculty requirements.

Course requirement	Evidence the team should prepare
Literature review	A cited comparison of relevant sources and the gap your experiments investigate.
Dataset quality and EDA	Shape/range/label checks, class counts, sample images and a written audit.
Feature engineering	Normalized flattened pixels and a justified HOG configuration.
Theoretical rigor	Explain each model, its assumptions, tuning choices and limitations.
Model application	Two representations with Logistic Regression, kNN, Random Forest and SVM.
GitHub and code quality	Reproducible code, clear README, saved settings, outputs and contribution evidence.
Presentation and individual viva	Every member presents their work and explains the shared experiment.
Project report	A coherent report in LaTeX/IEEE format, using the course template if supplied.

Scope boundary

Complete the classical ML component now. Save the CNN, pooling, dropout, CNN early stopping, AML-versus-CNN experiments and any implemented hybrid approach for Phase 2. A brief future-work paragraph is enough at this stage. The Phase 1 comparison is pixels versus HOG across four classical classifiers.

The attachment gives no Phase 1 evaluation date, page limit, presentation duration, grading weights, minimum accuracy, or required number of papers. Its September dates concern project/team selection. Check the course channel for later evaluation announcements; do not treat the proposed schedule here as an official deadline.

03 / SHARED FOUNDATIONS

Machine learning from zero

Here, **AML** means the classical/applied machine-learning part described in your project. You do not hand-write a rule saying "sleeves mean shirt." Instead, you show a model labeled examples so it can learn a rule from image measurements.

Term	Plain meaning in this project
Sample; label	One clothing image; its correct category number. The label is the answer.
Feature; representation	A numeric input. One image becomes either 784 pixel values or a HOG vector.
X and y	X is the feature matrix: one row per image. y is the matching list of labels.
Model; fit; predict	A learned decision rule; learn from X and y; assign labels to new X.
Parameter	Something learned during fitting, such as a Logistic Regression coefficient.
Hyperparameter	A setting chosen before fitting, such as k in kNN or C in an SVM.
Training / validation / test	Practice examples / development exam / final independent exam.
Overfitting	Doing very well on training data but failing to generalize to held-out data.
EDA	Exploratory data analysis: inspect the data before deciding how to model it.
Reproducibility	Another person can repeat the process using the same data, code and settings.

Walk through a single image

A grayscale image is a small grid of brightness values. Divide its raw pixel values by 255 to put them on a 0-1 scale. Flatten the grid to a vector, or compute a HOG description of local edge directions. A fitted classifier then maps that vector to one of the category IDs. Map the ID to the correct name for display.

Your first learning exercise

Before your assigned work, use a notebook to print an array's shape, show one image, print its label, and call fit then predict on a tiny example. Explain aloud why a row of X must stay attached to the same row of y. If that connection is broken, model training becomes meaningless.

Important nuance: flattening does not erase pixel values or their recoverable ordering. It gives a conventional classifier no built-in local-neighborhood or translation structure. HOG encodes local geometry, but its feature rule is fixed before classifier training.

04 / SHARED FOUNDATIONS

One shared, fair experiment

Freeze this protocol before comparing models

- Keep the official training and test partitions separate. Create a stratified 80/20 split of the official training partition with random_state=42: 48,000 fit images and 12,000 validation images. Save the indices once; everyone reuses them.
- Use two fixed representations and four model families: pixels/HOG × Logistic Regression/kNN/Random Forest/SVM. This produces eight main comparison rows. Add one DummyClassifier reference row outside those eight.
- Train on the fit partition, choose settings using validation macro F1, and use validation accuracy as a tie-breaker. If still tied, choose the simpler or faster option using validation measurements. Log the rule before seeing test scores.
- Fit learned scalers only on fit data. A fixed divide-by-255 transform and fixed per-image HOG extraction do not learn dataset statistics. Use a Pipeline for StandardScaler plus the classifier.
- After freezing one configuration per row and naming the validation-selected overall model, evaluate the eight frozen models on the official test partition. Do not choose new hyperparameters or a new overall winner from test results.

Training size and compute policy

Default: all eight final models use the same 48,000-image fit partition and the same 12,000-image validation partition. Keep those fitted models for final testing; do not silently refit some models on all 60,000 training images. This preserves the same split for Phase 2.

If compute makes this infeasible, agree before final runs on one saved stratified 12,000-image subset drawn only from the 48,000 fit images. Use that identical subset for all eight rows, retain the same validation/test partitions, and label the table "12,000 fit images per model." Exploratory subset runs must never masquerade as full-size results.

Required result package

Report accuracy, macro F1, recall for all ten classes, training time, inference time with a stated protocol, and a meaningful model-complexity description. Explain when a classical model has no neural-network-style parameter count. Save all eight rows, including weak runs; discuss shirt/T-shirt/pullover confusion using observed examples.

Protocol choices on this page are recommendations for consistency. The official split and the stated project metrics come from your project description; the specific validation fraction, seed and selection rule are proposed here.

05 / YOUR WORK

Your ownership and first session

You make sure the team is solving the intended problem using trustworthy inputs. An apparently strong classifier is useless if images and labels are misaligned, a split is contaminated, or someone accidentally loads handwritten MNIST digits instead of Fashion-MNIST clothing.

What you personally build

- **src/data.py:** one agreed loader returning raw images, integer labels and stable original IDs.
- **data/split_indices.npz:** one saved stratified training/validation split. Also save common smoke IDs and, only if approved by the team, the shared compute-limited fit subset.
- **notebooks/01_data_eda.ipynb:** validation of image dimensions, labels, pixel values, missingness, class counts, duplicates and representative images.
- **docs/data_audit.md and figures/eda_*.png:** findings, source details, limitations and publication-readable figures. Write the dataset/EDA report section and its slides.

First 90 minutes

0-20 min: learn array shapes and indexing. An array of N images should have shape (N, 28, 28), while labels have shape (N,). **20-45 min:** load the official data and display one image with its category name. **45-65 min:** count categories and print the value range. **65-90 min:** ask a teammate to identify several displayed items to catch orientation or label-map mistakes.

Think in pairs: image plus answer

Every image must remain attached to its true label and original sample ID. If you shuffle, shuffle indices and use those indices to select all three arrays together. Never independently shuffle the image array and label array.

Your handoff is accepted when

Puneet loads the exact same IDs and sees the same upright images and labels. Nakul confirms the saved fit/validation indices are disjoint and reproducible. Neelanshu can map any later prediction back to the original image without guessing its order.

Suggested effort: 18-24 hours across learning, loader/splits, quality checks, EDA, report writing and review. Your role includes technical implementation, not just drawing charts.

06 / YOUR WORK

Load the right images and labels

Use the official Fashion-MNIST partitions [R1-R2]. The dataset has 60,000 training and 10,000 test grayscale images, each 28 × 28, in ten categories. Preserve raw uint8 images in the loader; normalize in the shared feature transform. A library may be used only for data loading without implementing deep learning.

```
import numpy as np
from torchvision.datasets import FashionMNIST

train_ds = FashionMNIST("data/raw", train=True, download=True)
test_ds = FashionMNIST("data/raw", train=False, download=True)
images = train_ds.data.numpy()       # raw uint8
labels = train_ds.targets.numpy()    # integer IDs
test_images = test_ds.data.numpy()
test_labels = test_ds.targets.numpy()
assert images.shape == (60000, 28, 28)
assert test_images.shape == (10000, 28, 28)
# Only structural test checks now; lock away test content.
```

Use the official label map exactly

ID	Category	ID	Category
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle boot

Inspect one training example

```
import matplotlib.pyplot as plt
names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
         "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
i = 0
plt.imshow(images[i], cmap="gray", vmin=0, vmax=255)
plt.title(f"official_train:{i} | {names[int(labels[i])]}")
plt.axis("off")
plt.show()
```

Avoid test-image browsing during development. Downloading and checking file shape is different from exploring test examples to improve the model. Save source URL, retrieval date and checksums where available. The loader should not silently resize, rotate or invert images.

If torchvision installation is troublesome, download the four official gzip IDX files from [R1], verify the published checksums, and parse their image/label headers. Use one loader path consistently across the team; do not combine data from an unverified re-upload.

07 / YOUR WORK

Make one split everyone reuses

The validation set is carved only from the official training data. Stratification keeps the category proportions similar. A random seed makes the split repeatable; saving actual indices prevents subtle differences between teammates and future software versions. [R11]

```
from sklearn.model_selection import train_test_split
from pathlib import Path
ids = np.arange(len(labels))
tr, va = train_test_split(
    ids, test_size=0.20, random_state=42, stratify=labels)
assert len(tr) == 48000 and len(va) == 12000
assert len(np.intersect1d(tr, va)) == 0
assert np.array_equal(np.sort(np.r_[tr, va]), ids)
Path("data").mkdir(exist_ok=True)
np.savez("data/split_indices.npz", train_idx=tr, val_idx=va)

# Save one tiny development path too; these are NOT final runs.
smoke_tr, _ = train_test_split(
    tr, train_size=2000, random_state=42, stratify=labels[tr])
smoke_va, _ = train_test_split(
    va, train_size=1000, random_state=42, stratify=labels[va])
np.savez("data/smoke_indices.npz",
         train_idx=smoke_tr, val_idx=smoke_va)
```

Verify membership, not just lengths

- Fit/validation index sets must be disjoint and their union must equal the official training IDs. Every index must be in range and unique within its partition.
- Print counts for each of the ten labels in the fit and validation partitions. Check proportions, not only the total number of images.
- Store a SHA-256 hash of the split file in the run manifest. Everyone reads this file instead of calling train_test_split independently.
- Keep test IDs in their own namespace: official_test:0 is different from official_train:0. Numeric row values alone do not establish sample identity.

Optional compute-limited subset

Only if the team adopts the budget profile, stratify a 12,000-image subset from tr and save its IDs separately. Never draw it from va or from the test partition. All eight final comparison rows must use the same chosen fit IDs. Preserve the full split so Phase 2 can use it too.

Deterministic splitting is not leakage prevention by itself. A pipeline can use the correct indices and still leak information if its learned scaler is fitted on validation or test rows. See [R5].

08 / YOUR WORK

Audit dataset quality methodically

EDA shows what the data looks like; a quality audit asks whether it is safe to use. Record observed facts after running each check. Do not write "no missing values" or "no duplicates" simply because this is a standard benchmark.

Check	How to check; what to record
Shape and type	Print images.shape, labels.shape and dtypes. Require one label per image and the expected dimensions.
Finite/range	Require finite pixel values, raw values in 0-255 and integer label IDs in 0-9. Note constant or all-black images if any.
Missing/corrupt data	Check missing values and failed file reads. Inspect suspicious images; do not silently delete them.
Class counts	Use np.bincount(labels, minlength=10). Repeat on fit and validation. Plot counts with readable category names.
Exact duplicates	Hash raw image bytes inside the official training partition. Record duplicate groups and any conflicting labels.
Split independence	Check ID overlap. Distinct IDs can still contain identical pixels; quantify fit/validation exact-image overlap separately.
Orientation and map	Display several training examples per class. Check that category IDs, labels and upright display agree.
Real-world limits	The benchmark is small grayscale clothing imagery; production catalogues may differ in background, color, resolution and pose.

A simple raw-data audit

```
assert images.dtype == np.uint8
assert labels.ndim == 1 and len(labels) == len(images)
assert np.isfinite(images).all()
assert images.min() >= 0 and images.max() <= 255
assert set(np.unique(labels)) == set(range(10))
print("fit:", np.bincount(labels[tr], minlength=10))
print("val:", np.bincount(labels[va], minlength=10))
```

A value check does not prove semantic label correctness. A visually ambiguous shirt might be labeled shirt or T-shirt/top. Document uncertainty without casually relabeling official examples. If a genuine loading error is found, fix the loader and invalidate the affected downstream artifacts.

09 / YOUR WORK

Duplicates, orientation and decisions

Check exact duplicate images without losing labels

```
import hashlib
from collections import defaultdict
groups = defaultdict(list)
for i, image in enumerate(images):
    key = hashlib.sha256(image.tobytes()).hexdigest()
    groups[key].append(i)
dupes = [g for g in groups.values() if len(g) > 1]
conflicts = [g for g in dupes
             if len(set(labels[g].tolist())) > 1]
train_set, val_set = set(tr.tolist()), set(va.tolist())
cross = [g for g in dupes
         if any(i in train_set for i in g)
         and any(i in val_set for i in g)]
print("duplicate groups:", len(dupes))
print("conflicting-label groups:", len(conflicts))
print("fit/validation shared-image groups:", len(cross))
```

These counts are generated by your run; no value is assumed here. Exact hashing does not detect near-duplicates, almost identical silhouettes, or semantically repeated products. Describe that limit rather than claiming the dataset has no related examples.

What to do if overlap exists

Preserve the official test partition and labels. Record any training-partition duplicate issue in the audit. If fit/validation duplicate overlap is substantial, discuss a group-aware validation sensitivity analysis with the team before freezing results. All models must use the same resulting split. Keep the original fixed-split baseline if comparability requires it, and clearly separate any sensitivity study.

Do not inspect test duplicates as a shortcut to tuning, remove hard test examples, or silently deduplicate after models have been compared. A frozen test audit after model selection may reveal a limitation; report it without optimizing against it.

What "fix orientation" means

It means verify that your loader and display convention preserve the intended upright image. Do not arbitrarily rotate every garment or add rotations as "cleaning." HOG should see the same consistent orientation as the pixel baseline. If a loader produces transposed images, correct that bug for every partition and record the correction.

Decision log entry

Use: issue; detection method; number affected; example original IDs; decision; justification; downstream artifacts affected; reviewer. A deliberate "retain and disclose" decision is acceptable when justified. Unrecorded cleaning is not.

10 / YOUR WORK

Make EDA figures that answer questions

Use only training-derived data for exploratory choices. Each figure needs a question, a clear partition label and a caption stating what was observed. Avoid filling the report with decorative charts that do not influence understanding.

Figure	How to create it; why it matters
Class-count bar chart	Count fit and validation labels; label axes and use all ten class names. It verifies the stratified split.
Sample gallery	Show 3-5 seeded examples per class with original IDs. It reveals appearance variation and confirms labels/orientation.
Mean image per class	Average fit images within a category. Compare silhouette tendencies; do not call a mean image a real example.
Pixel-intensity histogram	Use a documented fit sample or all fit pixels. State whether values are raw 0-255 or normalized 0-1.
Similar-category panel	Place shirt, T-shirt/top and pullover fit examples beside each other. Describe overlap as a hypothesis before model evaluation.

Starter gallery

```
rng = np.random.default_rng(42)
fig, axes = plt.subplots(10, 3, figsize=(6, 16))
for cls in range(10):
    eligible = tr[labels[tr] == cls]
    chosen = rng.choice(eligible, size=3, replace=False)
    for ax, i in zip(axes[cls], chosen):
        ax.imshow(images[i], cmap="gray", vmin=0, vmax=255)
        ax.set_title(f"{names[cls]} | ID {i}", fontsize=8)
        ax.axis("off")
fig.tight_layout()
fig.savefig("figures/eda_gallery.png", dpi=180)
plt.close(fig)
```

Create figures/ before saving. The gallery is tall by design; split it into two panels if labels become too small in the report. For a two-column IEEE page, use a full-width figure or fewer examples with readable text.

Good caption structure

"Seeded examples from the fit partition show variation in sleeve length and garment outline. Shirt/T-shirt/top/pullover examples share some silhouettes, motivating later confusion analysis." This is a visual observation and hypothesis. Do not claim a model confuses those classes until Neelanshu measures the confusion counts.

11 / YOUR WORK

Write and hand off the data evidence

Structure your data audit

- **Provenance:** source, retrieval method, date, files/checksums and loader version. State that this is Fashion-MNIST, not digit MNIST.
- **Observed checks:** shapes, types, ranges, finite values, category counts and duplicate findings. Provide the notebook cell or function that generated each result.
- **Partitioning:** saved file path, seed, stratification, exact sizes, IDs and split hash. Note any budget fit subset separately.
- **Decisions:** what was retained or corrected and why. Describe dataset limitations and any remaining uncertainty.
- **Handoff:** function signatures, return order, sample-ID namespace, example usage and expected shapes.

Suggested loader contract

```
def load_official(split):
    """split is 'train' or 'test'.
    Returns:
      images: uint8 ndarray, shape (N, 28, 28)
      labels: integer ndarray, shape (N,)
      ids: integer ndarray, original row indices
    No shuffling, resizing, scaling, HOG, or fitting here.
    """
    ...

def load_indices(path="data/split_indices.npz"):
    """Return saved original IDs for fit and validation."""
    ...
```

Use exceptions with clear messages when a shape or file is invalid. Avoid printing thousands of sample rows. The loader should be independent of plotting so model owners can import it without opening charts.

Your report contribution

Write the dataset and EDA sections with actual counts and two to four informative figures. Explain why label IDs are not ordinal measurements: category 9 is not "more clothing" than category 0. Explain the difference between fixed 0-1 scaling and learned standardization, although Puneet implements the feature pipeline.

Review exercise

Give Puneet three original training IDs. Ask them to display the images using their feature input loader. Compare the IDs, label values and untransformed images. Give Neelanshu those same IDs and confirm their error-analysis function finds the correct originals. This catches an important class of silent alignment bugs.

12 / YOUR WORK

Troubleshooting and your completion test

Symptom	Likely cause; next action
Images look like digits	Wrong dataset loader or path. Stop downstream runs and load FashionMNIST explicitly.
Images are sideways	Transpose/reshape/display bug. Compare raw output to official examples; fix the same way everywhere.
High scores, wrong gallery labels	Potential row misalignment or wrong label map. Trace original IDs before trusting any score.
Normalized image nearly black	Possibly divided by 255 twice or used display limits 0-255 on 0-1 data. Inspect min/max and display scale.
Counts or shapes differ by member	Different source/split/subset or stale files. Compare split hashes, source files and train_n.
HOG fails on a sample	Wrong image shape, unexpected color/channel dimension or non-finite values. Check raw loader contract first.
Memory usage spikes	Large unnecessary copies or converting entire arrays to float64. Preserve raw uint8 and transform in batches.

Your final checklist

- The loader returns the correct shapes and types; all labels are mapped correctly; raw values remain unchanged until the feature stage.
- The shared split is saved, disjoint, reproducible and reviewed. Smoke indices and any budget subset are visibly separate from final-size indices.
- Quality findings are based on executed checks. You have not silently removed duplicates, relabeled ambiguous images or explored test content to guide modeling.
- Every figure has a saved path, readable labels, a seed where sampling is used, a partition name and a factual caption.
- Your audit/report text states benchmark limitations and gives another member enough information to reproduce the data preparation.

Explain these concepts without notes

Why stratify? Why save indices? Why does splitting by ID not prove pixel uniqueness? Why is label alignment essential? Why must the test partition remain separate? What is the difference between an EDA hypothesis and a measured model failure?

Your concrete completion standard

A teammate can load your data and saved split, reproduce the EDA checks, and trace any prediction to the exact original image and label.

13 / TEAM DELIVERY

Handoffs and shared file contract

Each person works on their own files, commits meaningful changes, and opens a review request or asks a teammate to review. Share generated evidence with file paths and the exact run ID; a screenshot in a group chat is insufficient.

Sender → receiver	What is handed over; acceptance check
Sudeshhna → everyone	Loader, labels, split_indices.npz, audit.md. Row counts and train/validation IDs agree.
Puneet → Samarth + Neelanshu	Feature functions, HOG settings, feature metadata. A sample ID yields the same vector everywhere.
Puneet + Samarth → Neelanshu	Fitted pipelines, validation predictions, run logs and feature metadata. Every prediction has a split and sample ID.
Neelanshu → Nakul + model owners	Verified tables, error figures, timing method and audit findings. Results trace back to saved predictions.
All four → Nakul	Their report sections, figure captions, references, contribution notes and slides.
Nakul → everyone	Integrated notebook, draft report and slides. Each member checks their section and practices the full demo.

Use these identifiers consistently

sample_id: original row number within the official partition, plus **source_split** = official_train or official_test. Train/validation IDs both refer to official_train. Never use the row number of a shuffled feature cache as the original sample ID.

Prediction CSV: run_id, stage, source_split, sample_id, y_true, y_pred. stage is val or test. Do not invent probabilities when a classifier only exposes decision scores. Preserve deterministic sample order.

Run record: run_id, owner, git_commit, seed, split_file_hash, train_n, val_n, feature_name, feature_dim, feature_config, scaler, estimator, hyperparameters, package_versions, fit_seconds, status, warnings and artifact_paths. Keep validation and test metrics in separate fields/files.

Results CSV: run_id, stage, train_n, feature, model, accuracy, macro_f1, fit_seconds, predict_ms_per_image, end_to_end_ms_per_image, serialized_bytes, complexity_type, complexity_value. Put per-class recall in a separate ten-row table per run. Use seconds, milliseconds and bytes explicitly.

Every handoff includes: what changed, how to run it, expected shapes, the evidence checked, and known limitations. The person receiving it must rerun a small example before accepting it.

14 / TEAM DELIVERY

Individual viva practice

Practice without reading the answers. Explain each idea simply, then open the relevant code or figure as evidence. These are suggested talking points, not answers to memorize word for word.

Everyone must answer these

What is the difference between validation and test?
Validation guides development. Test measures the frozen procedure on held-out examples. Looking at test failures and then tuning would contaminate that final assessment.

Why use both pixels and HOG?
They test whether a fixed description of local edges helps compared with raw intensities. HOG may help some classifiers and hurt others; the experiment must establish the result.

Why is accuracy alone insufficient?
It summarizes total correctness and can hide a weak category. Macro F1 averages class F1 values equally; per-class recall shows which categories are missed.

What can you honestly conclude about a CNN now?
Only that a CNN is a motivated Phase 2 direction. There is no evidence that your future CNN outperforms your ML baseline until that comparison is run.

Sudeshhna: questions about your own work

Why save IDs if the seed is fixed?
Saved IDs define actual membership and order across environments. They also let us trace predictions back to original images and compare artifact hashes.

What did your quality audit establish?
State the checks you actually ran and their findings. Distinguish finite values and shapes from semantic label correctness and near-duplicate detection.

What if duplicate images cross fit and validation?
Report the overlap, preserve the official test set, and consider a clearly separated group-aware validation sensitivity study before freezing decisions.

Five-minute live demonstration

Show your own file and explain one nontrivial choice. Run a small cell or load a saved artifact. Trace one image from its original ID through features to a prediction. Read a validation metric correctly. Explain one failure, one limitation and one improvement reserved for Phase 2. Everyone must also describe the eight-row experiment and the shared split.

15 / TEAM DELIVERY

Suggested 10-day work plan

This is a relative plan, not an official deadline. Budget roughly 18-26 focused hours per person plus unattended compute; beginners may need longer. If you have fewer days, reduce the optional search grid and parallelize member-owned work. Keep data integrity, all eight baseline rows, and honest reporting.

Day	Your focus	Team gate
1	Learn arrays; load one image	All five understand scope and vocabulary
2	Implement loader and checks	Environment works; literature notes started
3	Save split and sample IDs	Data/split/feature contracts agreed
4	Finish EDA and data audit	One tiny run reaches the evaluator
5	Review Puneet's feature inputs	Protocol and compute budget fixed
6	Support full-run data issues	Eight comparable rows progressing
7	Verify row order and metadata	Configurations frozen; test run authorized internally
8	Write dataset/EDA report text	Verified figures and individual text ready
9	Rehearse EDA and data viva	One full rehearsal; report compiles
10	Verify data instructions from scratch	Fresh-run check; contribution log complete

Definition of done for every member

- Your files run in the agreed environment, your inputs and outputs are documented, and a teammate has reviewed one small run.
- Your report section explains what you did, why you did it, what evidence supports it, and what its limits are. Captions and citations are included.
- Your contribution log names real files, commits, experiments and reviews. It records actual work, not an artificial equal number of commits.
- You can explain your own implementation and the shared project during an individual viva. You have practiced a backup demo using saved outputs.

If a run fails, preserve the error and settings. Tell the team early and revise the compute plan consistently. Do not fabricate results, omit failed rows without explanation, or quietly change the split.

16 / TEAM DELIVERY

Sources and how to use them

Course source: CSAI_Project_Instructions_Phase_1_and_Phase_2.md, supplied by you. Its Phase 1 list and individual-viva rule govern the required work. Your pasted Project 11 description supplies the feature/model comparison and metrics. Neither source contains actual results for your team.

Technical references below were checked on 27 October 2026. Stable documentation can change; record the versions actually installed for your experiments. Reference IDs such as [R3] in this guide point to these links. Read and cite original material in your submitted report; this guide is not a substitute for a literature review.

[R1] Fashion-MNIST official repository: data files, label map and official partitions

[R2] Xiao, Rasul and Vollgraf (2017), Fashion-MNIST dataset paper

[R3] scikit-image: HOG API and settings

[R4] scikit-image: HOG worked explanation

[R5] scikit-learn: leakage and preprocessing pitfalls

[R6] scikit-learn: LogisticRegression

[R7] scikit-learn: KNeighborsClassifier

[R8] scikit-learn: RandomForestClassifier

[R9] scikit-learn: SVC

[R10] scikit-learn: model evaluation and metrics

[R11] scikit-learn: stratified train_test_split

[R12] Torchvision: FashionMNIST loader

[R13] Dalal and Triggs (2005): original HOG paper, CVPR

[R14] scikit-learn: LinearSVC alternative and multiclass behavior

[R15] LeCun et al. (1998): author-hosted document-recognition paper record

Suggested research reading beyond API pages

Read Dalal and Triggs (2005), "Histograms of Oriented Gradients for Human Detection" [R13], for the original descriptor. Its task is human detection, so do not transfer its scores to clothing classification. LeCun et al. (1998), "Gradient-Based Learning Applied to Document Recognition" [R15], is useful Phase 2 background; it does not require a Phase 1 CNN implementation.

Citation discipline

For each source record title, authors, year, task, representation, model, evaluation protocol and relevance. Separate a paper's results from your results. Repository leaderboard entries are not automatically controlled comparisons or verified targets. Use actual bibliographic metadata in your LaTeX .bib file; never cite an unread paper as evidence for a specific finding.

Code blocks are starter recipes to adapt and run in your project environment. They have not been run on Fashion-MNIST as part of creating these guides. Toy calculations are illustrative, not measured project performance.